



VampiFox

APPLICATION FRAMEWORK

MODULAR • API-FIRST • MULTI-DOMAIN • GOLANG

 ERP  Akademia  Medika  People  Sosial

Architecture Vision v0.2 — Expanded Edition

"The Night Never Sleeps, The Fox Never Rests"

1. New Vision — From ERP to Application Framework

VampiFox started as an ERP Framework, but its vision has evolved into something more fundamental: a Golang-based Application Framework capable of supporting various business information system domains, not just ERP.

This change is not just a feature expansion — it is a philosophical shift. VampiFox no longer asks "how to build a better ERP than Odoo?", but rather "how to build a foundation that can support any enterprise-quality information system?".

1.1 Old vs New Vision Comparison

Dimension	v0.1 — ERP Framework	v0.2 — Application Framework
Target Users	Trading & manufacturing companies	Businesses, schools, hospitals, clinics, foundations, NGOs
Domain	Single domain: ERP	Multi-domain: ERP, Akademia, Medika, People, Sosial
Frontend	Built-in admin panel (HTMX)	API-first: free to use React, Vue, Angular, mobile, anything
Database	PostgreSQL (schema-per-tenant)	GORM multi-driver: PostgreSQL, MySQL, SQL Server, SQLite
Sub-framework	None	VampiFox ERP, Akademia, Medika, People, Sosial
Positioning	"Go-based alternative to Odoo/ERPNext"	"Foundation for any enterprise information system"

1.2 Unchanged Principles

Although the scope has expanded, the three foundational pillars of VampiFox remain the same:

Pillar	Keyword	Meaning in Practice
--------	---------	---------------------

 MODULAR	Go packages	Each domain and feature is an independent module that can be enabled or disabled without affecting others
 API-FIRST	REST + GraphQL	The framework does not dictate the UI. Frontend, mobile, IoT — all are free to use as long as they communicate via API
 DX-FIRST	foxctl + zero-config	Developers can be productive in 30 minutes. The framework helps, not hinders

2. VampiFox Sub-Framework Ecosystem

VampiFox Core is the engine. Sub-frameworks are the vehicles built on top of it. Each sub-framework is a cohesive collection of Go modules for a specific domain. Developers can use a single sub-framework or combine multiple ones simultaneously.



VampiFox ERP

Business & Manufacturing Information System

Module	Module	Module	Module
Accounting & GL	Inventory Management	Purchasing (PO)	Sales & AR
Fixed Assets	Warehouse Management	Manufacturing & BOM	CRM
Point of Sale	Multi-Currency	Financial Reports	e-Commerce Connector



VampiFox Akademia

School, Campus & Learning Management Information System

Module	Module	Module	Module
Student Data	Student Admissions	Curriculum & Subjects	Schedules & Classes
Attendance	Grading & Report Cards	Tuition Payments	LMS (Materials & Assignments)

Forums & Discussions	Online Exams	Digital Library	Alumni Management
----------------------	--------------	-----------------	-------------------

VampiFox Medika

Hospital Information System (SIMRS), Clinic Management & Medical Records

Module	Module	Module	Module
Patient Registration	Electronic Medical Records	Outpatient & ER	Inpatient & Wards
Pharmacy	Laboratory	Radiology	Digital Queuing
Billing & Insurance	BPJS Connector	Doctor Schedules	Hospital Reports & Statistics

VampiFox People

HRIS / HRMS — Human Resource Management System

Module	Module	Module	Module
Employee Data	Recruitment & Selection	Onboarding	Organizational Structure
Attendance & Shifts	Leave & Permits	Payroll	Income Tax 21 & BPJS
KPI & Performance Appraisal	Training & Certification	Digital Payslips	HR Reports

VampiFox Sosial

Foundation, NGO & Social Institution Management

Module	Module	Module	Module
Member & Donor Data	Programs & Activities	Donations & Zakat	Fund Management
Foundation Financial Reports	Volunteer Management	Beneficiaries	Social Impact Reports

Sub-Framework Combinations

Developers are free to combine sub-frameworks according to their needs. Example: A private hospital can use Medika + People + ERP (for hospital accounting). A foundation-based school can use Akademia + Sosial + People. All run on a single VampiFox instance with multi-tenancy.

3. Database Strategy — GORM Multi-Driver

VampiFox uses GORM as the sole ORM with multi-driver support. Developers are free to choose the RDBMS that best suits their infrastructure, budget, and team expertise — without having to change a single line of business code.

3.1 Supported Databases

Database	Driver	Status	Recommended Use Case
PostgreSQL 14+	gorm.io/driver/postgres	✓ Primary	Large-scale production, enterprise multi-tenant, JSONB & full-text features
MySQL 8+ / MariaDB	gorm.io/driver/mysql	✓ Supported	Shared hosting, legacy infrastructure, teams familiar with MySQL
SQL Server 2019+	gorm.io/driver/sqlserver	✓ Supported	Windows Enterprise, integration with Microsoft ecosystem
SQLite 3	gorm.io/driver/sqlite	✓ Supported	Local development, demos, embedded apps, testing
MongoDB	—	✗ No	VampiFox only supports RDBMS — NoSQL is out of scope
Redis	—	⚠ Cache Only	Only as a cache layer (Shadow), not primary storage

Design Decision: RDBMS Only

Business systems require ACID transactions, relational integrity, and complex queries across tables. MongoDB and other NoSQL databases excel in different use cases (documents, time-series, graph). VampiFox focuses on domains that truly need RDBMS — and executes them exceptionally well.

- ACID compliance — financial transactions, stock, and medical records cannot have partial failures
- Relational integrity — foreign keys between patients, doctors, and medical records must be maintained at the DB level
- Complex JOINS — business reports often require cross-querying 5-10 tables at once

- Mature tooling — pgAdmin, DBeaver, SSMS, MySQL Workbench are already familiar to clients' IT teams
- Easy audit trails — WITH SYSTEM VERSIONING or triggers are more reliable in RDBMS

3.2 How Developers Choose a Database

```
// configs/vampifox.yaml
fangs:
  driver: postgres # postgres | mysql | sqlserver | sqlite
  host: localhost
  port: 5432
  user: vampifox
  password: secret
  dbname: vampifox_main

# For MySQL:
# driver: mysql
# host: localhost
# port: 3306

# For SQL Server:
# driver: sqlserver
# host: MYSERVER\SQLEXPRESS
# port: 1433
```

VampIFox Den will read the configured driver and automatically initialize GORM with the correct driver. No code changes are required.

3.3 Multi-Tenancy per Database Driver

Driver	Tenant Isolation	Strategy	Notes
PostgreSQL	✅ Schema per tenant	SET search_path = vfx_{slug}	Best isolation, recommended
MySQL	✅ Database per tenant	USE vfx_{slug}	MySQL does not have schemas, uses databases
SQL Server	✅ Schema per tenant	USE [vfx_{slug}]	Schema inside a single database

SQLite	⚠ File per tenant	Separate file connections	Only for dev/single-user apps
--------	-------------------	---------------------------	-------------------------------

4. API Strategy — REST + GraphQL

VampiFox adopts an API-first architecture. The framework does not dictate the UI — anything that speaks HTTP can be a VampiFox client, ranging from React web apps, Vue dashboards, Flutter mobile apps, to external systems and IoT devices.

4.1 Two API Layers

Layer	Endpoint	Best For	Characteristics
REST API	/api/v1/*	CRUD operations, mobile apps, system integrations	Predictable, cacheable, simple, universal
GraphQL	/graphql	Complex dashboards, flexible queries, BFF pattern	Fetch exactly what you need, single roundtrip

4.2 REST API Design

```

GET    /api/v1/{module}/{resource}      # list with pagination
GET    /api/v1/{module}/{resource}/:id  # single record detail
POST   /api/v1/{module}/{resource}      # create new
PUT    /api/v1/{module}/{resource}/:id  # full update
PATCH /api/v1/{module}/{resource}/:id  # partial update
DELETE /api/v1/{module}/{resource}/:id  # delete (soft delete)

```

ERP Examples:

```

GET    /api/v1/accounting/invoices?page=1&limit=20&status=unpaid
POST   /api/v1/accounting/invoices
GET    /api/v1/inventory/items?warehouse_id=abc&low_stock=true

```

Akademia Examples:

```

GET    /api/v1/akademia/students?class_id=X-IPA-1&year=2025
POST   /api/v1/akademia/attendance/bulk

```

Medika Examples:

```
POST    /api/v1/medika/patients/:id/medical-records
GET     /api/v1/medika/queue?date=today&poly=umum
```

4.3 Standard Response Envelope

All VampiFox REST responses wrap data in a consistent envelope:

```
// Success Response
{
  "success": true,
  "data": { ... },           // or array []
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 354,
    "total_pages": 18
  },
  "request_id": "vfx-01J..." // for tracing
}

// Error Response
{
  "success": false,
  "error": {
    "code": "INVOICE_NOT_FOUND",
    "message": "Invoice with the provided ID not found",
    "field": null           // filled if validation error
  },
  "request_id": "vfx-01J..."
}
```

4.4 GraphQL — For Complex Queries

VampiFox's GraphQL endpoint is built with gqlgen (code-first, type-safe). Suitable for frontends that need data from multiple resources simultaneously without multiple roundtrips.

```
# Example: Hospital Dashboard – fetch today's stats in 1 query
query DashboardRS {
  todayStats {
    totalPatients
    newRegistrations
    bedOccupancyRate
  }
}
```



```

}
queue(date: "today", poly: UMUM) {
  number, patientName, status, estimatedTime
}
lowStockMedicines(threshold: 10) {
  name, currentStock, unit, reorderPoint
}
}

# Example: School Dashboard
query DashboardSekolah($classId: ID!) {
  class(id: $classId) {
    name
    students { id, name, attendanceRate }
    upcomingExams { subject, date, room }
    averageGrade
  }
}

```

4.5 Frontend & Mobile Freedom

Platform	Applicable Technologies	Notes
Web — Admin Panel	React , Vue 3 , Angular , Svelte , Next.js , Nuxt	Use REST for CRUD, GraphQL for dashboards
Web — Public Portal	Next.js (SSR) , Nuxt , SvelteKit , Astro	SSR for SEO, fetch data from REST API
Mobile — Native	Swift (iOS) , Kotlin (Android)	Consume REST API with JWT auth
Mobile — Cross	Flutter , React Native , Expo	Recommended for mobile team efficiency
Desktop	Electron , Tauri , Flutter Desktop	Suitable for POS cashiers or offline-capable clinic apps
IoT / Embedded	Any HTTP client	For sensor, RFID, barcode scanner integration
External Systems	REST API + Webhook	Other ERPs, marketplaces, payment gateways

No Frontend Lock-in

VampiFox has no opinion on the frontend. There is no "mandatory official UI kit". Developers can build an admin panel with any technology, or buy commercial UI templates and connect them to the VampiFox API.

5. Core Technical Architecture

This section explains the internal architecture of VampiFox Core — the foundational layers supporting all sub-frameworks.

5.1 Layer Architecture

CLIENT LAYER				
React	Vue	Angular	Flutter	Mobile System
API GATEWAY LAYER				
REST /api/v1/*		GraphQL /graphql		
Bloodgate (Auth)		Covenant (RBAC)		
TenantResolver		RateLimiter	CORS	
MODULE LAYER				
ERP	Akademia		Medika	
People	Sosial		Custom Modules	
CORE SERVICE LAYER				
Auth/JWT	RBAC	Tenant	Audit	Events
Scheduler	Webhook	Mailer	Storage	Queue
DATA LAYER (Fangs)				
GORM ORM — PostgreSQL / MySQL / SQL Server				
Shadow Cache — Redis				
Storage — MinIO / S3				

5.2 Full Request Flow

HTTP Request: POST /api/v1/medika/patients

Header: Authorization: Bearer <JWT>

Header: X-VampiFox-Tenant: rs-maju-sehat

1. TenantResolver → resolve "rs-maju-sehat" → load Tenant from Shadow/Fangs
2. Bloodgate → validate JWT, extract BloodClaims
3. Covenant → check "medika:patient.create" permission
4. Handler → validate request body
5. Service → business logic (check duplicates, generate Medical Record number)
6. Repository → GORM INSERT to schema vfx_rs_maju_sehat
7. NightVision → emit trace + metrics
8. Event Bus → publish "patient.created" event
9. Webhook Engine → send payload to registered endpoints (async)
10. Response → 201 Created + new patient data

5.3 Module Registration

How developers combine sub-frameworks and custom modules in a single instance:

```
package main

import (
    "github.com/vampifox/vampifox/internal/den"

    // Sub-frameworks
    "github.com/vampifox/erp/accounting"
    "github.com/vampifox/erp/inventory"
    "github.com/vampifox/medika/patients"
    "github.com/vampifox/medika/pharmacy"
    "github.com/vampifox/people/payroll"

    // Company custom modules
    "github.com/rsmajusehat/custom/bpjs"
)

func main() {
    app := den.NewDen(den.ConfigFromFile("configs/vampifox.yaml"))

    app.RegisterModules(
        // From VampiFox Medika
        patients.New(),
```

```

        pharmacy.New(),

        // From VampiFox People
        payroll.New(),

        // From VampiFox ERP (hospital accounting)
        accounting.New(),
        inventory.New(),    // for medicine & medical equipment stock

        // Specific custom module for this hospital
        bpjs.New(),          // BPJS Kesehatan integration
    )

    app.Awaken(ctx)
}

```

6. Architecture Decision Records — Update v0.2

The following ADRs are new decisions and revisions from v0.1 based on the expanded scope.

ADR-004: Database: GORM Multi-Driver, RDBMS Only

Status: ACCEPTED

Context: Developers from various industries (manufacturing, healthcare, education) have different database preferences and infrastructures.

Decision: VampiFox supports all popular RDBMS via GORM drivers. NoSQL (MongoDB, Cassandra, etc.) is explicitly not supported as primary storage.

Reason for RDBMS only: The business domains served by VampiFox (ERP, SIMRS, HRIS, academic) all require ACID transactions, referential integrity, and complex JOINS. NoSQL does not offer significant advantages for these domains and actually complicates audit trails and financial reporting.

ADR-005: API: REST + GraphQL, No Mandatory Server-side Rendering

Status: ACCEPTED

Revision of: ADR-002 v0.1 which set the HTMX admin panel as the only frontend

Decision: VampiFox Core is a pure API backend. REST for standard operations, GraphQL for complex queries. No frontend is mandatory or bundled with the framework.

Regarding the admin panel: VampiFox will provide a reference implementation of an admin panel as a separate project (vampifox-admin), built with React. This is not part of the Core and is not mandatory to use.

ADR-006: Brand: VampiFox + Sub-Framework per Domain

Status: ACCEPTED

Context: With a scope covering ERP, education, healthcare, HRIS, and foundations — the name "VampiFox ERP Framework" no longer reflects the true vision.

Decision: The main brand remains "VampiFox" as an Application Framework. The tagline changes from "ERP Framework" to "Application Framework". Sub-frameworks are given thematic names reflecting their respective domains.

Sub-Framework	Domain	Tagline
VampiFox ERP	Business & Manufacturing	"Run Your Business, Not Your Software"
VampiFox Akademia	Education & LMS	"Empower Every Learner"
VampiFox Medika	Healthcare & SIMRS	"Care for Patients, Not Paperwork"
VampiFox People	HRIS / HRMS	"Your People, Your Greatest Asset"
VampiFox Sosial	Foundation & NGO	"Mission-Driven, Data-Informed"

7. Roadmap — Updated v0.2

Phase 0 — Nightfall: Core Foundation (Now → 3 months)

The foundation that supports all sub-frameworks. No coupling to specific domains is allowed.

- Den — DI container, lifecycle, config loader multi-driver
- Fangs — GORM abstraction layer (PostgreSQL, MySQL, SQL Server, SQLite)
- Shadow — Redis cache with Shadow Cascade
- Sanctum + Bloodgate — JWT auth
- Covenant — RBAC with field-level permission
- TenantResolver — multi-tenancy per database driver
- NightVision — structured logging + OpenTelemetry
- Moonphase — tenant-aware scheduler
- Webhook Engine — event-driven integration
- REST API skeleton + GraphQL skeleton
- foxctl: new, dev, migrate, scaffold module

Phase 1 — First Blood: VampiFox ERP + People (3 → 10 months)

The first most needed sub-frameworks.

- VampiFox ERP: Accounting (GL, Invoice, Payment, AP/AR)
- VampiFox ERP: Inventory (Item, Warehouse, Stock Movement)
- VampiFox ERP: Purchasing & Sales
- VampiFox People: Employee, Attendance, Leave
- VampiFox People: Payroll + PPh 21 + BPJS
- Automated REST API documentation
- Client SDK for JavaScript/TypeScript (for frontend developers)
- Reference admin panel: vampifox-admin (React + Tailwind)

Phase 2 — Bloodline: Akademia + Medika (10 → 20 months)

Expansion into education and healthcare domains.

- VampiFox Akademia: Student, Class, Schedule, Attendance
- VampiFox Akademia: Grading, Report Card, SPP Payment
- VampiFox Akademia: LMS (Course, Material, Quiz, Assignment)
- VampiFox Medika: Patient, Registration, Queue
- VampiFox Medika: Medical Record (EMR), Diagnosis, SOAP
- VampiFox Medika: Pharmacy, Laboratory, Billing
- VampiFox Medika: BPJS Connector
- Flutter SDK for mobile development

Phase 3 — Eternal Night: Sosial + Advanced (20+ months)

Platform maturity, AI integration, and complete ecosystem.

- VampiFox Sosial: Foundation, Donation, Beneficiary, Impact Report
- VampiFox ERP: Manufacturing (BOM, Work Order, Production)
- NightMind — AI/automation engine (workflow builder, anomaly detection)
- Mobile SDK: React Native, Flutter
- Marketplace module (source-based, build from source)
- Helm charts + Kubernetes operator
- VampiFox Cloud — managed hosting

Cross-Domain Feature Priorities

Feature	Priority	Applies To
Native multi-tenancy	P0	All sub-frameworks
REST API + GraphQL	P0	All sub-frameworks
Field-level RBAC	P0	All sub-frameworks
Automated audit trails	P1	ERP, Medika, Akademia
Webhooks & event system	P1	All sub-frameworks

Notifications (email, push, WA)	P1	All sub-frameworks
Reports & exports (PDF, Excel)	P1	All sub-frameworks
Multi-currency	P2	ERP, Sosial
Digital signatures	P2	ERP, Medika, Akademia
AI workflow automation (NightMind)	P3	All sub-frameworks

8. VampiFox Glossary — Updated

Glossary updates to reflect VampiFox's new scope as a multi-domain Application Framework.

Term	Layer	Definition
Den	Core	Dependency injection container & lifecycle manager. The main nest where all services are born.
Fangs	Database	GORM multi-driver abstraction layer. "Bites" any database — Postgres, MySQL, SQL Server.
Shadow	Cache	Redis cache layer with Shadow Cascade — auto-invalidates cache when data changes.
Sanctum	Auth	JWT token manager. Only those carrying a "valid invitation" can enter.
Bloodgate	Middleware	Auth middleware — checks tokens on every request.
Covenant	Middleware	RBAC middleware — enforces permission hierarchies down to the field level.
Moonphase	Scheduler	Cron-based tenant-aware job scheduler. Every tenant has its own schedule.
NightVision	Observability	Structured logging + OpenTelemetry tracing + Prometheus metrics, all built-in.
NightMind	AI Layer	(Phase 3) AI/automation engine — workflow builder and anomaly detection.

BloodClaims	Auth	JWT payload carrying user identity: tenant, role, and permissions.
Bloodline	RBAC	Role and permission hierarchy. Overlord > Elder Vampire > Daywalker > Familiar > Spectre.
VampiFox ERP	Sub-framework	Collection of modules for business systems: accounting, inventory, purchasing, sales.
VampiFox Akademia	Sub-framework	Collection of modules for education: SIS, LMS, grading, library.
VampiFox Medika	Sub-framework	Collection of modules for healthcare: EMR, pharmacy, queueing, BPJS.
VampiFox People	Sub-framework	Collection of HRIS modules: employees, attendance, payroll, KPIs.
VampiFox Sosial	Sub-framework	Collection of foundation modules: donations, programs, beneficiaries.
foxctl	CLI	Developer CLI tool: scaffold, migrate, dev server, seed, health check.
Awaken	Lifecycle	Start server — "the sun sets, the vampire awakes".
Slumber	Lifecycle	Graceful shutdown — "dawn arrives, the vampire rests".
Territory	Tenant	Domain/subdomain controlled by a single tenant.
Bite	Database	Opening a database connection.
Haunt	Cache	Storing a value in the cache.
Recall	Cache	Retrieving a value from the cache.
Vanish	Cache	Deleting a value from the cache.